-- FICHIER: OPT_SRC/src/sem_phase/idl-sem_phase-univ_ops.adb PAGE: 1/1 DEBUT

```
with UNCHECKED_CONVERSION;¶separate(¬IDL.SEM_PHASE )¶--|----------------------------------------------------------------------------------------------------
---¶--|¬¬UNIV_OPS¶--|-------------------------------------------------------------------------------------------¶package body UNIV_OPS is¶   ¶
NUM_VAL¬¬: constant INTEGER¬:= NODE_NAME'POS( DN_NUM_VAL );¶   ¶  type UDIGIT_PAIR_TYPE¬is record¶¬¬¬  U1, U2¬: UDIGIT;¶¬¬¬end record;¶¬¬¬pragma PACK(
 UDIGIT_PAIR_TYPE¬);¶¬  ¶  type VECTOR_DIGITS_PAIRS¬is array(1..LINE_IDX(126)) of¬UDIGIT_PAIR_TYPE;¶¬¬¬pragma PACK( VECTOR_DIGITS_PAIRS );¶¶  type VEC
TOR_PAIRS¬¬is record¶¬¬¬  L¬: NATURAL;¬¬¬¬¬¬--| NOMBRE DE "CHIFFRES" 10_000 AIRES¶¬¬¬  S¬: UDIGIT;¬¬¬¬¬¬¬--| SIGNE¬+1 OR -1¶¬¬¬  P¬: VECTOR_DIGITS_PAIRS
;¬¬¬¬¬--| PAIRES DE CHIFFRES¶¬¬¬end record;¬¬¬pragma PACK( VECTOR_PAIRS );¶         ¶   ¶--|----------------------------------------------------------------
--------------------------------------¶--|¬¬PROCEDURE¬DIGIT_MUL¶--|¶procedure¬DIGIT_MUL¬( A, B :in UDIGIT; HIGH, LOW :out UDIGIT ) is¶  A1¬: UDIGIT¬:=
 A / 100;¶  B1¬: UDIGIT¬:= B / 100;¶  A2¬: UDIGIT¬:= A mod 100;¶  B2¬: UDIGIT¬:= B mod 100;¶  XX¬: UDIGIT¬:= A1 * B2 + A2 * B1;¶  LL¬: UDIGIT¬:= A2 *
B2 + (XX mod 100) * 100;¶begin¶  LOW  :=¬LL mod 10_000;¶  HIGH :=¬A1 * B1 +¬LL / 10_000 + XX / 100;¶end DIGIT_MUL;¶--|-------------------------------
-----------------------------------------------------------------¶--|¬¬FUNCTION DIGIT_DIV¶--|¶function DIGIT_DIV ( H, L, A :UDIGIT ) return UDIGIT is¶
 QUO¬: UDIGIT;¶  PH, PL¬: UDIGIT;¬¬¬-- TRIAL PRODUCT¶begin¶¬¬¬¬¬-- MUST HAVE H < A (OTHERWISE¬OVERFLOW)¬      ¶  if H = 0 then¶     return L / A;¶  elsif
 A¬< 100 then¶¬¬¬¬¬-- FORCE A >= 100¶    return DIGIT_DIV( H*100+L/100, (L mod 100)*100, A*100);¶  else¶¬¬¬¬¬-- ALWAYS¬REDUCE TO¬A SIMPLER¬CASE¶    if H
 >= 100 then¶      QUO¬:= (H / ((A+99) / 100)) * 100;¶    else¶      QUO¬:= ((H*100) + (L/100)) / ((A+99) / 100);¶    end if;¶¬¬¬¬¬-- ASSERT: QUO > 0¶
   DIGIT_MUL(A, QUO, PH, PL);¶¬¬¬¬¬-- ASSERT: H*10000 + L = QUO*A + PH*10000 + PL¶    if L >= PL then¶      return QUO + DIGIT_DIV(H - PH, L - PL, A);¶
    else¶      return QUO + DIGIT_DIV(H - 1 - PH, L + 10000 - PL, A);¶    end if;¶  end if;¶end DIGIT_DIV;¶--|||||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION U_INT¶--|¶function U_INT ( V :VECTOR ) return TREE is¶begin¶¶CAS_RETOUR_ELEMEN
T_HI_SHORT_CODE:¬¬¬¬¬¬¬¬--| QUAND¬LA VALEUR¬EST DANS -32768 .. 32767 ON REND UN TREE HI AVEC ABSS ET NSIZ¶  begin¶    if V.L = 1 then¬¬¬¬¬¬¬¬¬¬--| UN SE
UL UDIGIT : ENTIER DANS -10_000 .. 10_000¶      if V.D( 1 ) =¬0 then¶        return (HI,¬NOTY=> DN_NUM_VAL, ABSS=> 0, NSIZ=> 0 );¶      elsif V.S = +1
 then¶        return (HI,¬NOTY=> DN_NUM_VAL, ABSS=> POSITIVE_SHORT( V.D( 1 ) ), NSIZ=> 0 );¶      elsif V.S = -1 then¶        return (HI,¬NOTY=> DN_NU
M_VAL, ABSS=> POSITIVE_SHORT( V.D( 1 ) - 1 ), NSIZ=> 1 );¶      end¬if;¶¶    elsif¬V.L = 2 then¬¬¬¬¬¬¬¬¬¬--| 2 UDIGITS ET CAS DE L INTERVALLE -32768 ..
32767¶      if V.S = -1 and then ( V.D( 2 ) =¬3 and V.D( 1 ) = 2_768 ) then¬¬¬¬¬--| VALEUR -32_768¶        return (HI,¬NOTY=> DN_NUM_VAL, ABSS=> 32_767
, NSIZ=>1 );¶      end¬if;¶      if V.D( 2 ) <= 2¬¬¬¬¬¬¬¬¬--| VALEURS ABSOLUES DANS 0 .. 29_999¶        or else ( V.D( 2 ) =¬3 and V.D( 1 ) <= 2_767 )
¬¬¬¬¬¬--| OU VALEURS ABSOLUES DANS 30_000 .. 32_767¬¶      then¶        if V.S = +1¬then¬¬¬¬¬¬¬¬¬--| VALEURS POSITIVES¶¬return (HI, NOTY=> DN_NUM_VAL,¶¬
¬ABSS=> POSITIVE_SHORT( V.D( 2¬) * 10_000 + V.D( 1¬) ), NSIZ=> 0 );¶        elsif V.S =¬-1 then¬¬¬¬¬¬¬¬¬¬--| VALEURS NEGATIVES¶¬return (HI, NOTY=> DN_NU
M_VAL,¶¬¬ABSS=> POSITIVE_SHORT( V.D( 2¬) * 10_000 + V.D( 1¬) - 1 ), NSIZ=> 1 );¶        end if;¶      end¬if;¶    end if;¶  end CAS_RETOUR_ELEMENT_HI_
SHORT_CODE;¶¶CAS_RETOUR_BLOC_UDIGITS:¬ ¶  declare¶    W_LEN¬¬: ATTR_NBR¬:= ATTR_NBR( (V.L+1) / 2 );¬¬¬¬¬--| NOMBRE DE MOTS (PAIRES DE¬UDIGITS) DU BLOC¶
    R¬¬: TREE¬¬:= MAKE( DN_NUM_VAL, W_LEN );¬¬¬¬¬--| CREER¬LE NOEUD¶  begin¶¶PORTER_LE_SIGNE:¶    declare¶      ENTETE¬: TREE¬:= DABS( 0, R );¬¬¬¬¬¬¬--|
 PRENDRE L ENTETE DU NOEUD¬DN_NUM_VAL¶    begin¶      if V.S = -1 then ENTETE.ABSS := 1;¬¬¬¬¬¬¬--| SIGNE¬NEGATIF INDIQUER DANS LE ABSS¶      else ENTET
E.ABSS := 0;¶      end¬if;¶      DABS( 0, R, ENTETE );¬¬¬¬¬¬¬¬¬--| REPORTER DANS L¬ENTETE¶    end PORTER_LE_SIGNE;¶¶COPIE_PAIRES_UDIGITS:¶    declare¶
     ROUND_LEN¬¬: ATTR_NBR¬:= ATTR_NBR( V.L / 2 );¶      VDP¬¬¬: VECTOR_PAIRS;¬for VDP use at V'ADDRESS;¶      function CAST_TREE¬is new UNCHECKED_CON
VERSION( UDIGIT_PAIR_TYPE, TREE );¶    begin¶      for¬I in 1..ROUND_LEN loop¶        DABS( I, R,¬CAST_TREE( VDP.P( I¬) ) );¬¬¬¬¬¬¬--| COPIER UNE PAIRE
 VDP.P(I)¶      end¬loop;¶¶      if V.L mod 2 = 1 then¬¬¬¬¬¬¬¬¬--| NOMBRE IMPAIR DE MOTS¶        DABS( ROUND_LEN+1, R,¬CAST_TREE( (V.D( V.L ), 0) ) );¬
¬¬¬--| COPIER LE DERNIER UDIGIT SUIVI DE 0¶      end¬if;¶    end COPIE_PAIRES_UDIGITS;¶    ¶    return R;¶  end CAS_RETOUR_BLOC_UDIGITS;¶end U_INT;¶--
||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION U_REAL¶--|¶function U_REAL ( NUMER, D
ENOM :VECTOR ) return TREE is¶  R¬: TREE¬:= MAKE( DN_REAL_VAL );¶begin¶  D( XD_NUMER, R, U_INT( NUMER ) );¶  D( XD_DENOM, R, U_INT( DENOM ) );¶  retur
n R;¶end U_REAL;¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION U_REAL¶--|¶functio
n U_REAL ( NUMER, DENOM :TREE ) return TREE is¶  N_SPREAD, D_SPREAD¬: VECTOR;¶  THE_REAL¬¬: TREE¬:= MAKE( DN_REAL_VAL );¶begin¶  SPREAD(¬NUMER, N_SPRE
AD );¶  SPREAD(¬DENOM, D_SPREAD );¶      ¶  if D_SPREAD.L = 2¬and then D_SPREAD.D( 1 ) = 0 and then D_SPREAD.D(¬2 ) = 0 then¬--| DENOMINATEUR NUL¶
return TREE_VOID;¶  end if;¶¶  N_SPREAD.S := N_SPREAD.S * D_SPREAD.S;¶  D_SPREAD.S := +1;¶  V_LOWEST_TERMS( N_SPREAD, D_SPREAD );¶  D( XD_NUMER, THE_R
EAL, U_INT( N_SPREAD ) );¶  D( XD_DENOM, THE_REAL, U_INT( D_SPREAD ) );¶  return THE_REAL;¶end U_REAL;¶--||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬PROCEDURE¬SPREAD¶--|¶procedure¬SPREAD ( T :TREE; V¬:in out VECTOR ) is¶  BAD_NSIZ, CANNOT_S
PREAD, BAD_SIGN¬: exception;¶begin¶  if T.PT¬= HI then¬¬¬¬¬¬¬¬¬¬--| CAS DE L ENTIER¬COURT -32768 .. 32767¶    if T.NOTY = DN_NUM_VAL then¬¬¬¬¬¬¬¬--| ASS
URER LE BON TYPE DE NOEUD¶      declare¶        IVAL¬: INTEGER;¶      begin¶        if    T.NSIZ = 0  then IVAL :=¬INTEGER( T.ABSS );¬¬¬¬¬¬--| INDICATE
UR DE SIGNE POSITIF¶        elsif T.NSIZ = 1 then¬IVAL := -INTEGER( T.ABSS ) - 1;¬¬¬¬¬--| INDICATEUR DE SIGNE NEGATIF¶        else raise BAD_NSIZ;¶
    end if;¶        SPREAD( IVAL, V );¶      end;¶      else raise CANNOT_SPREAD;¶    end if;¶          ¶  else¬¬¬¬¬¬¬¬¬¬¬--| CAS DU POINTEUR¬VERS BLOC¬
DE UDIGITS¶    if T.PT /= P or¬else T.TY¬/= DN_NUM_VAL then¬¬¬¬¬¬¬--| ASSURER LE TYPE¬POINTEUR ET NOEUD¶      raise CANNOT_SPREAD;¶    end if;¶¶PREPARE
_VECTOR_FROM_BLOCK:¶    declare¶      ENTETE¬: TREE¬¬:= DABS( 0, T );¬¬¬¬¬¬--| ENTETE CONTIENT¬LE SIGNE DANS ABSS MOD 2 ET LE NOMBRE DE PAIRES DANS NSI
Z¶      VDP¬¬: VECTOR_PAIRS;¬for VDP use at V'ADDRESS;¶      function CAST_UDIGIT_PAIR¬is new UNCHECKED_CONVERSION( TREE, UDIGIT_PAIR_TYPE );¶    begi
n¶      V.L¬:= 2 * NATURAL( ENTETE.NSIZ );¶      if ENTETE.ABSS mod 2 = 1 then V.S¬:= -1;¬¬¬¬¬¬¬--| NOMBRE INDIQUE NEGATIF¶      elsif ENTETE.ABSS mod
2¬= 0 then V.S := +1;¬¬¬¬¬¬¬--| SINON¬SIGNE POSITIF¶      end¬if;¶      for¬I in 1 ..¬ENTETE.NSIZ loop¶        VDP.P( I ) := CAST_UDIGIT_PAIR(¬DABS( I,

 T ) );¶      end¬loop;¶    end PREPARE_VECTOR_FROM_BLOCK;¶ ¶    NORMALIZE( V );¶  end if;¶¶exception¶  when BAD_NSIZ=>¬      ¶    PUT_LINE( "!! SPREA
D NSIZ NOT 0/1 =" & ATTR_NBR'IMAGE( T.NSIZ ) );¬¬¬--| DANS UN DN_NUM_VAL COURT NSIZ CONTIENT L INDICATION DE SIGNE¶    raise¬PROGRAM_ERROR;¶  when CAN
NOT_SPREAD=>¶    declare¶      NN¬: NODE_NAME;¶    begin¶      case T.PT is¶        when HI¬=> NN := T.NOTY;¶        when P | L¬=> NN := T.TY;¶        when
S¬=> NN := DN_SOURCELINE;¶      end¬case;¶      PUT_LINE( "!! CANNOT SPREAD " & NODE_NAME'IMAGE( NN )¬);¶      raise PROGRAM_ERROR;¶    end;¶end SPREA
D;¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬PROCEDURE¬SPREAD¶--|¶procedure¬SPREAD ( I
 :INTEGER; V :in out VECTOR )¬is¶  VAL_POS¬¬: INTEGER¬¬:= abs( I¬);¶begin¶  if I < 0 then V.S¬:= -1;¬¬¬¬¬¬¬¬¬¬--| ENTIER NEGATIF SIGNE NEGATIF¶  else V.
S := +1;¬¬¬¬¬¬¬¬¬¬--| SIGNE¬POSITIF¶  end if;¶¶  V.D(1) := UDIGIT(¬VAL_POS mod 10_000 );¬¬¬¬¬¬¬--| 0 .. 1E4 -1¶  V.L := 1;¶¶  if VAL_POS < 100_000_000 t
hen¬¬¬¬¬¬¬¬--| 0 .. 1E8 -1¶    V.D(2) := UDIGIT( (VAL_POS / 10_000) mod 10_000 );¶    V.L := 2;¶  else¶    V.D(2) := UDIGIT( (VAL_POS / 100_000_000) mo
d¬10_000 );¶    V.L := 3;¬¬¬¬¬¬¬¬¬¬--| TROIS¬UDIGITS¶  end if;¶    ¶  NORMALIZE( V );¶end SPREAD;¶--|||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬PROCEDURE¬NORMALIZE¶--|¶procedure¬NORMALIZE¬( V :in out VECTOR ) is¶begin¶  if V.L > 252 then¶
  PUT_LINE ( "!! UNIV INTEGER > 1E756 - COMPILER LIMITATION" );¶    raise¬PROGRAM_ERROR;¶  end if;¶¶ENLEVE_ZEROS_NON_SIGNIF:¬¶  while V.L > 1 and¬V.D(
 V.L ) = 0 loop¶    V.L := V.L - 1;¶  end loop ENLEVE_ZEROS_NON_SIGNIF;¶end NORMALIZE;¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||¶--|¬¬PROCEDURE¬V_ADD¶--|¶procedure¬V_ADD ( A¬:VECTOR; R :in out VECTOR ) is¶  ALEN¬: INTEGER¬:= A.L;¶  TEMP¬: UDIGIT;¶
  CARRY¬: UDIGIT¬:= 0;¶begin¶  if R.L < A.L then¶    for I¬in R.L + 1 .. A.L loop¶      R.D( I ) := A.D( I );¶    end loop;¶    ALEN := R.L;¶    R.L :
= A.L;¶  end if;¶          ¶  R.L := R.L + 1;¶  R.D( R.L ) := 0;¶  for I in 1 .. ALEN loop¶    TEMP := A.D( I ) + R.D( I¬) + CARRY;¶    if TEMP >= 10_0
00 then¶      CARRY := 1;¶      TEMP := TEMP - 10_000;¶    else¶      CARRY := 0;¶    end if;¶    R.D( I ) := TEMP;¶  end loop;¶          ¶  for I in A
LEN + 1¬.. R.L loop¶    exit when CARRY¬= 0;¶    TEMP := R.D( I ) + CARRY;¶    if TEMP >= 10_000 then¶      TEMP := TEMP - 10_000;¶      CARRY := 1;¶
   else¶      CARRY := 0;¶    end if;¶    R.D( I ) := TEMP;¶  end loop;¶          ¶  NORMALIZE( R );¶end V_ADD;¶--||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬PROCEDURE¬V_SUB¶--|¶procedure¬V_SUB ( A¬:VECTOR; R :in out VECTOR ) is¬¬--| ON SUPP
OSE |A| <= |R| !!!¶  TEMP¬: UDIGIT;¶  BORROW¬: UDIGIT := 0;¶begin¶  if A.L = R.L and then A = R¬then¶    R.L := 1;¶    R.D( 1 ) := 0;¶  end if;¶
  ¶  for I in 1 .. A.L¬loop¶    TEMP := R.D( I ) - A.D( I¬) - BORROW;¶    if TEMP < 0 then¶      BORROW := 1;¶      TEMP := TEMP + 10_000;¶    else¶
    BORROW := 0;¶    end if;¶    R.D( I ) := TEMP;¶  end loop;¶          ¶  for I in A.L + 1 .. R.L loop¶    exit when BORROW = 0;¶    TEMP := R.D( I
) - BORROW;¶    if TEMP < 0 then¶      TEMP := TEMP + 10_000;¶      BORROW := 1;¶    else¶      BORROW := 0;¶    end if;¶    R.D( I ) := TEMP;¶  end l
oop;¶          ¶  NORMALIZE(R);¶end V_SUB;¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬PR
OCEDURE¬V_MUL¶--|¶procedure¬V_MUL ( A, B :VECTOR; R :in out VECTOR ) is¶  H, L, TEMP, CARRY¬: UDIGIT;¶  K¬¬: INTEGER;¶begin¶  R.S := +1;¶  R.L := A.L
+ B.L;¶  if R.L > 252 then¶    PUT_LINE ( "!! UNIV PRODUCT TOO LARGE");¶    raise¬PROGRAM_ERROR;¶  end if;¶          ¶  for I in 1 .. R.L¬loop¶    R.D(
 I ) := 0;¶  end loop;¶          ¶  for I in 1 .. A.L¬loop¶    for J¬in 1 .. B.L loop¶      K := I + J - 1;¶      DIGIT_MUL ( A.D( I ), B.D( J ), H, L
);¶      TEMP := R.D( K ) + L;¶      if TEMP >= 10_000 then¶        CARRY := 1;¶        TEMP := TEMP - 10_000;¶      else¶        CARRY := 0;¶      en
d¬if;¶¶      R.D( K ) := TEMP;¶      K := K + 1;¶      TEMP := R.D( K ) + H + CARRY;¶¶      if TEMP >= 10_000 then¶        CARRY := 1;¶        TEMP :=
 TEMP - 10_000;¶      else¶        CARRY := 0;¶      end¬if;¶      R.D( K ) := TEMP;¶¬      ¶      while CARRY >¬0 loop¶        K¬:= K + 1;¶        TEM
P := R.D( K ) + CARRY;¶        if TEMP >= 10_000 then¶¬CARRY := 1;¶¬TEMP := TEMP - 10_000;¶        else¶¬CARRY := 0;¶        end if;¶        R.D( K )
:=¬TEMP;¶      end¬loop;¶    end loop;¶  end loop;¶          ¶  NORMALIZE ( R );¶end V_MUL;¶--|||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||¶--|¬¬PROCEDURE¬V_SCALE¶--|¶procedure¬V_SCALE (¬A :INTEGER; R :in out VECTOR ) is¶  H, L¬: UDIGIT;¶  CARRY¬:
UDIGIT¬:= 0;¶begin¶  for I in 1..R.L loop¶    DIGIT_MUL ( UDIGIT( A ), R.D( I ), H, L );¶    L := L + CARRY;¶    R.D( I ) := L mod 10_000;¶    CARRY¬:
= H + L/10_000;¶  end loop;¶          ¶  if CARRY > 0 then¶    R.L := R.L + 1;¶    R.D( R.L ) := CARRY;¶  end if;¶end V_SCALE;¶--||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬PROCEDURE¬V_DIV¶--|¶procedure¬V_DIV ( A¬:VECTOR; R, Q :in out VECTOR ) is¬
¬--| A EST¬LE DIVISEUR, R LE DIVIDENDE DEVIENT RESTE, Q EST LE QUOTIENT¶¬      -- A POOR LONG-DIVISION¬ALGORITHM¬(EXTRA ITERATIONS)¶¬      -- GOOD ENO
UGH FOR GOVERNMENT WORK, FOR NOW¶  A_TRIAL¬¬: constant UDIGIT := A.D(A.L)¬+ 1;¬¬¬--| CHIFFRE LE PLUS¬SIGNIFICATIF DU DIVISEUR +1¶  QDIG, CARRY, TEMP¬:
UDIGIT;¶  R_PREC¬¬: VECTOR;¶  PPROD¬¬: VECTOR;¶begin¶      ¶  if A.D(¬1 ) = 0 and then A.L = 1 then¬¬¬--| SI LE¬PREMIER CHIFFRE EST¬NUL ET LE¬NOMBRE D
E¬CHIFFRES EST 1 ¶    PUT_LINE ( "V_DIV: DIVIDE BY ZERO" );¬¬¬¬--| DIVISEUR NUL¶    raise¬PROGRAM_ERROR;¶  end if;¶      ¶  R_PREC := R;¶  Q.S := +1;¶
 declare¶    DIFF¬: INTEGER¬:= R.L - A.L + 1;¶  begin¶    if DIFF <= 0 then¶      Q.L¬:= 1;¶    else¶      Q.L¬:= R.L - A.L + 1;¶    end if;¶  end;¶
       ¶  for I in 1 .. Q.L¬loop¶    Q.D(I) := 0;¶  end loop;¶        ¶  while not V_LESS ( R, A ) loop¬¬¬--| TANT QUE LE RESTE EST SUPERIEUR OU EGAL
 AU DIVISEUR¶    if R.L = A.L¬¬¬¬--| RESTE¬ET DIVISEUR ONT MME NOMBRE DE CHIFFRES¶      and then R.D( R.L ) = A.D( A.L )¬¬¬--| ET MME CHIFFRE¬LE PLUS
 SIGNIFICATIF¶    then¬¬¬¬¬¬--| ON SAIT QUE LE RESTE EST >= AU DIVISEUR¶      QDIG := 1;¬¬¬¬¬--| LA DIVISION DES¬CHIFFRES LES PLUS SIGNIFICATIFS DONNERA
¬1¶    R.L¬:= R.L + 1;¬¬¬¬¬--| ALLONGER LE RESTE¶      R.D( R.L ) :=¬0;¬¬¬¬¬--| POUR UN ZERO NON SIGNIFICATIF¶    else¬¬--| ILS N'ONT PAS MME NOMBRE¬D
E CHIFFRES OU PAS MME CHIFFRE LE PLUS¬SIGNIFICATIF¶      if R.D( R.L )¬>= A_TRIAL then¬¶        R.L := R.L + 1;¶        R.D( R.L ) := 0;¶      end¬if;
¶      QDIG := DIGIT_DIV ( R.D( R.L ), R.D( R.L-1 ), A_TRIAL¬);¶    end if;¶¬  ¶    CARRY¬:= QDIG;¶¬  ¶    for I¬in R.L - A.L .. Q.L¬loop¶      TEMP :
= Q.D( I ) + CARRY;¶      if TEMP < 10_000 then¶        Q.D(I) := TEMP;¶        exit;¶      end¬if;¶      Q.D( I ) := TEMP - 10_000;¶      CARRY := 1;
¶    end loop;¶¬  ¶    R := R_PREC;¶    V_MUL¬( Q, A, PPROD );¬--| QUOTIENT * A¶    if V_LESS ( R, PPROD ) then¶      raise PROGRAM_ERROR;¶    end if;
¶¶    V_SUB¬( PPROD, R );¬--| RETIRE DU RESTE¬PRECEDENT¶  end loop;¶  NORMALIZE( Q );¶end V_DIV;¶--||||||||||||||||||||||||||||||||||||||||||||||||||

||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬PROCEDURE¬V_REM¶--|¶procedure¬V_REM ( A¬:VECTOR; R :in out VECTOR ) is¶   INUTILE¬: VECTOR;¶begin
¶  V_DIV (¬A, R, INUTILE );¶end V_REM;¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬PROCED
URE¬V_GCD¶--|¶procedure¬V_GCD ( A, B :VECTOR; R :in out VECTOR ) is¶  S¬: VECTOR¬:= B;¶begin¶  R := A;¶  loop¶    V_REM¬( R, S );¶    if S.L = 1 and t
hen S.D( 1 ) = 0 then¶      return;¶    end if;¶    V_REM¬( S, R );¶    if R.L = 1 and then R.D( 1 ) = 0 then¶      R := S;¶      return;¶    end if;¶
  end loop;¶end V_GCD;¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶--|¬¬PROCEDURE¬V_LOWEST_TE
RMS¶--|¶procedure¬V_LOWEST_TERMS ( A,¬B :in out¬VECTOR ) is¶  GCD¬: VECTOR;¶  TREM¬: VECTOR;¶  SIGN¬: UDIGIT;¶begin¶  if B.L = 1 and then B.D( 1 ) = 1
 then¶    return;¶  end if;¶  if A.L = 1 and then A.D( 1 ) = 0 then¶    B.L := 1;¶    B.D( 1 ) := 1;¶    return;¶  end if;¶  SIGN :=¬A.S * B.S;¶  V_GC
D (¬A, B, GCD¬);¶  if GCD.L > 1 or else GCD.D(1) > 1 then¶    TREM := A;¶    V_DIV¬( GCD, TREM, A );¶    TREM := B;¶    V_DIV¬( GCD, TREM, B );¶  end
if;¶  A.S := SIGN;¶  B.S := +1;¶end V_LOWEST_TERMS;¶--|#######################################################################################
##¶--|¬¬FUNCTION V_EQUAL¶--|¶function V_EQUAL ( A, B :VECTOR ) return BOOLEAN is¬¬¬¬¬¬--| COMPARAISON EN VALEUR ABSOLUE¶begin¶  if A.L /= B.L then¶
return FALSE;¶  else¶    return A.D(1..A.L) = B.D(1..B.L);¶  end if;¶end V_EQUAL;¶--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||¶--|¬¬FUNCTION V_LESS¶--|¶function V_LESS ( A, B :VECTOR ) return¬BOOLEAN is¬¬¬¬¬¬--| NE PREND PAS EN¬COMPTE LE¬SIGNE¶b
egin¶  if A.L /= B.L then¶    return A.L < B.L;¶  end if;¶  for I in reverse 1 .. A.L loop¶    if A.D(I) /= B.D(I) then¶      return A.D(I)¬< B.D(I);¶
    end if;¶  end loop;¶  return FALSE;¶end V_LESS;¶   ¶--|------------------------------------------------------------------------------------------
------¶end UNIV_OPS;¶
```

-- FICHIER: OPT_SRC/src/sem_phase/idl-sem_phase-univ_ops.adb PAGE: 1/1 FIN